Stack

Concept of ADT : Revision :

So far, we have discussed that a type is an interface – it states what we can and does not tell us how it does it. We have also said that a language cannot provide all possible types. Some languages provide a mechanism to make out own type. In others, we can simulate it.

We have also said that there are different ways of arranging information. Each is geared for a particular purpose. These are called data structures. A data structure is an abstract data type. It tells the user what operations it supports. It does not commit to any particular implementation. A client or the user has the flexibility of selecting from various vendors supporting that interface. A developer can support several clients.

We have also realized that the interface is sacred and that it should not be changed. We can change the implementation without telling the user. In languages like 'C' and C++, the client may have to recompile the code in the worst case.

Stack : ADT :

- Is a data structure
- Has zero or more components
- Can operate at only one end – normally called the stack top
- Supports LIFO : Last In First Out
- What goes in last, comes out first
- Does not support the concept of traversal

- **Interface** :
    - Create
    - Destroy
    - Push : add an element at the top
    - Pop : remove an element at the top
    - Peek : check the top element without removing it
    - Is_empty : check whether the stack is empty
    - Is_full : check whether the stack is full

- **Implement** using
    - A linked list
    - An Array
-

Applications of stack :

Let us discuss a few applications of the stack.

1. **Stack frame or activation record** : When a function is called, an activation or a stack frame is created. The parameters, the local variables, the temporary variables the compiler creates and the return address are stored in this piece of memory. If the called function calls another, another activation record is created on top of this. When the function returns, the activation record is removed.
2. **Think : Would every stack frame have the same size? How to we remove it?**

3. **Redo-undo operations** : Do you remember pressing <ctrl>z in most of the editors? To support such an operation, the system should track the operations applied. It should undo the last operation first. So, it requires a stack.

4. **Parentheses matching**: To overcome the issue with the fixed precedence and association of operators and to change the order of evaluation, we use parentheses. The compiler will have to check whether the parentheses are well formed. Not only the number of left parentheses should match the number of right parentheses, but the left one should appear

before the right. We use a stack to check whether the right parenthesis in the expression matches the left one on the top of the stack.

You may remember that we studied 4 types of brackets in our high school days.

1. Vinculum
2. Simple brackets or parentheses
3. Curly or flower brackets
4. Square or big brackets.
5.

If a language were support all but the first – first cannot be supported directly – can we use the stack to check whether such an expression is well formed?

5. **Expression Evaluation**: We normally write expressions with operators in between the operands. This is called infix expression. The evaluation of operators depends on both the precedence and association of operators. Would it be possible to create an expression where all the operators are at the same level of precedence and have left to right association? In such a case, we can evaluate the expression as soon as the operator is reached. Such an expression is postfix expression where the operator appears after the operator. We can convert an infix expression to a postfix expression by using a stack of operators and we can evaluate an expression by using a stack of operands.
   We will discuss this at length later in this course.

   There are lots of applications using stack. We will mention them. You will come across them later in your study.

   1. Depth first search of a graph
   2. Backtracking
   3. Branch and bound
   4. Simulation of recursive functions

      …

Some interesting diagrams for you to view:

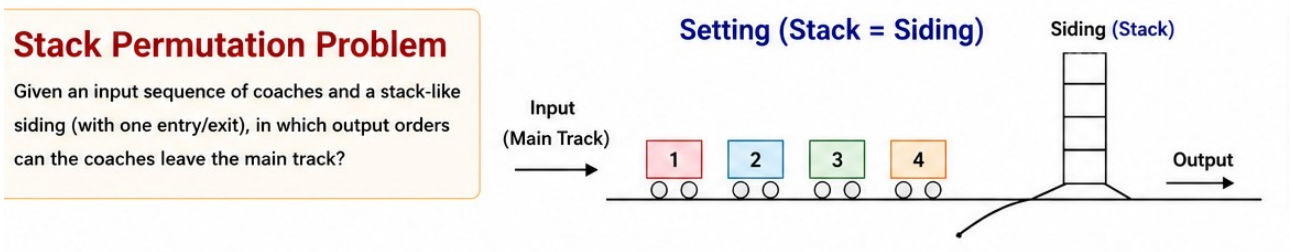


I am sure you would have seen this in restaurants. You can pull the top place or push a plate at the top.

An interesting problem for you to solve. There are a few coaches on the track. The order can be changed by using a sliding stack. There are three possible actions.

1. Coach goes directly to the output track.
2. Coach is pushed into the sliding stack
3. Coach is popped out from the sliding stack.

At the end, all coaches are popped out.



Stack Implementation

Let us implement the stack in two ways.

1. Stack using an array
2. Stack using a list

Array implementation of the stack:

Interface:

Let us capture the interface in the stack.h file.

We will create an ADT called stack_t. We will support the following functions on this type.

Please refer to stack.h in the folder stack_using_array.

This will initialize the stack.

```
void init_stack(stack_t *ptr_stack);
```

This will remove all the elements from the stack

```
void deinit_stack(stack_t *ptr_stack);
```

We are implementing a stack of character. We could of course choose any other valid type of component. The function push will add a character at the top of the stack.

```
void push(stack_t *ptr_stack, char ch);
```

The function pop removes the top character from the stack and returns it.

```
char pop(stack_t *ptr_stack);
```

We cannot pop if the stack is empty. We can check the condition using the function is_empty.

```
int is_empty(stack_t *ptr_stack);
```

We cannot push if the stack is full. We can check the condtion using the function is_full.

```
int is_full(stack_t *ptr_stack);
```

In the last two exceptional cases, the way of handling depends on the user. We are not discussing this at present.

Implementation:

We will implement the stack as having an array of the component type. Unfortunately, this has to be part of the header file. This forces the client to recompile his code if this is changed. But he does not have to modify his code.

The interface would remain same even if we implement the stack using a linked list but the structure list_t.

Let us look at the stack.h file.

```
#define MAXSIZE 5
struct stack
{
        char key_[MAXSIZE];
        int top_;
};
typedef struct stack stack_t;
```

Observe that apart from the array of component type – in our case character, we have another field top_. This holds the index of the top element.

Either this integer field could be the # of elements in the stack or the index of the top element. Based on this all functions will change. We shall track the index of the top element of the stack using the variable top_.

The functions are implemented in stack.c file.

Initialize stack :

void init_stack(stack_t *ptr_stack);

This function initializes top_ to -1. This is an invalid index on the array. Every variable we create should be associated with some meaning. In this case, top_ is the index of the last element added and is not the index of the position where the next element can be added. This decision affects all the functions we implement.

Client Program:

We have written a simple program to test this stack implementation. The file is client.c

We create and initialize a stack of characters. We read a string from the keyboard. We traverse the string and push each character on to the stack. Then we pop the characters and display until the stack becomes empty.

So, we will get the input string in the reverse order.

We shall discuss the list implementation in the next.

Deinitialize stack:

void deinit_stack(stack_t *ptr_stack);

This will reinitialize the top_ to -1 indicating that the stack has conceptually become empty.

Push a character into the stack:

void push(stack_t *ptr_stack, char ch);

This will increment the top_ and then check whether there is space by calling is_full. If there is no space, it gives a message. Otherwise, the character is placed on the top of the stack.

Pop a character from the stack:

char pop(stack_t *ptr_stack);

This function checks whether the stack is not empty. If empty, this function gives a message, otherwise, takes the top element out, decrements the top_ and returns the element taken out.

int is_empty(stack_t *ptr_stack);

This checks whether the stack is empty by comparing top_ with -1. This function should be used by pop before using top_.

int is_full(stack_t *ptr_stack);

As we are implementing the stack using an array, it is possible that at some time the array becomes full. This function compares the value of top_ and the array size to determine whether the stack is full. This function should be used before pushing into the stack.

List implementation of the stack:

Interface:

All the function declarations remain the same. We do not want the client to modify his code. At the most he may have to recompile the code.

So, the functions are :

```
void init_stack(stack_t *ptr_stack);
void deinit_stack(stack_t *ptr_stack);
void push(stack_t *ptr_stack, char ch);
char pop(stack_t *ptr_stack);
int is_empty(stack_t *ptr_stack);
int is_full(stack_t *ptr_stack);
```

We will change the structure of stack to have a pointer to a node where the node has a character key_ and a pointer to a node next_ - node is a self referential structure. This change requires that the client needs to recompile the code.

```
struct node
{
        char key_;
        struct node* next_;
};
typedef struct node node_t;

struct stack
{
        node_t* top_;
};
typedef struct stack stack_t;
```

implementation of stack using the list:

Most of these functions are fairly straightforward.

Init_stack :

This will initialize top_ to become NULL indicating that the stack is empty.

void init_stack(stack_t *ptr_stack);

deinit_stack :

This has not been implemented. This requires that we pop nodes repeatedly until the stack becomes empty.

void deinit_stack(stack_t *ptr_stack);

push:

This function will check whether the stack is not yet full. If so, creates a new node, populates it and then adds to the stack. The new node created will have its next_ field point to wherever top_ points to and then top_ is changed to point to the new node created.

You may observe that the order of these operations matters.

void push(stack_t *ptr_stack, char ch);

pop:

This function whether the stack is empty. If not, it returns the key_ in the top element and also removes it. There are a few steps. **The order of steps matters**.

- Copy the key_ to a local variable
- Create a pointer to point to the top element – make the variable have the value of top_
- Change top_ to point the next node in the stack
- Free the node to which the local pointer points to
- Return the key_ copied to a local variable

char pop(stack_t *ptr_stack);

is_empty:

This function checks whether top_ is NULL.

int is_empty(stack_t *ptr_stack);

is_full:

This function is tricky. How to make whether the heap has memory to allocate? There is no simple way. We will try allocating calling malloc. If it fails, is_fill returns true – that is 1. If it succeeds, this function returns false – that is 0. But we should free the memory allocated. Otherwise garbage is created leading to memory leak.

```
int is_full(stack_t *ptr_stack);
```

So, we have a type called stack implemented in two ways. The client can select whichever he wants. Is this not so wonderful?